Introduction to Test Automation in Business Central

"Testing can show the presence of errors, but not their absence."

- Edsger W. Dijkstra

By Partners, For Partners
DIRECTIONS
NORTH AMERICA

April 16 -19, 2023 Hyatt Regency Grand Cypress, Orlando, FL

About Me

- Microsoft Dynamics NAV / D365 Business Central Enthusiast
- Development, System Architecture, and more
- 1998
- brad@tigunia.com

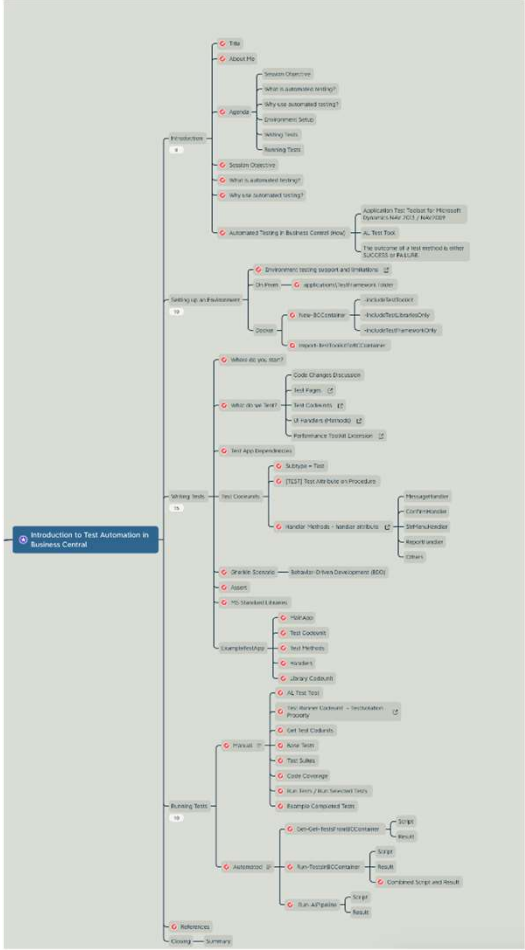
Brad Prendergast

Agenda

- Objective
- What is automated testing?
- Why use automated testing?
- Setting up an Environment
- Writing Tests
- Running Tests

Session Objective

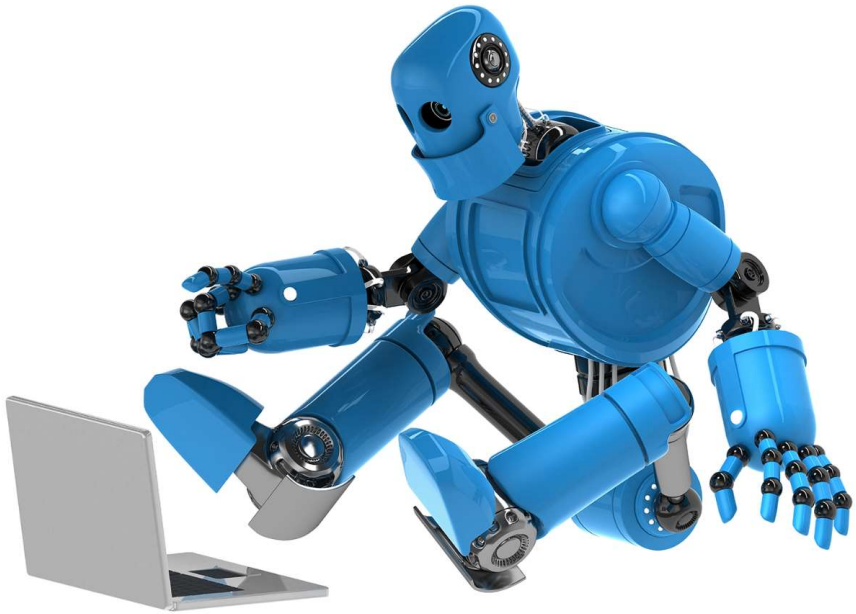
- Basic Understanding of Automated Testing in Business Central
  - Environment Setup
  - Writing Tests
  - Running Tests



What is Automated Testing

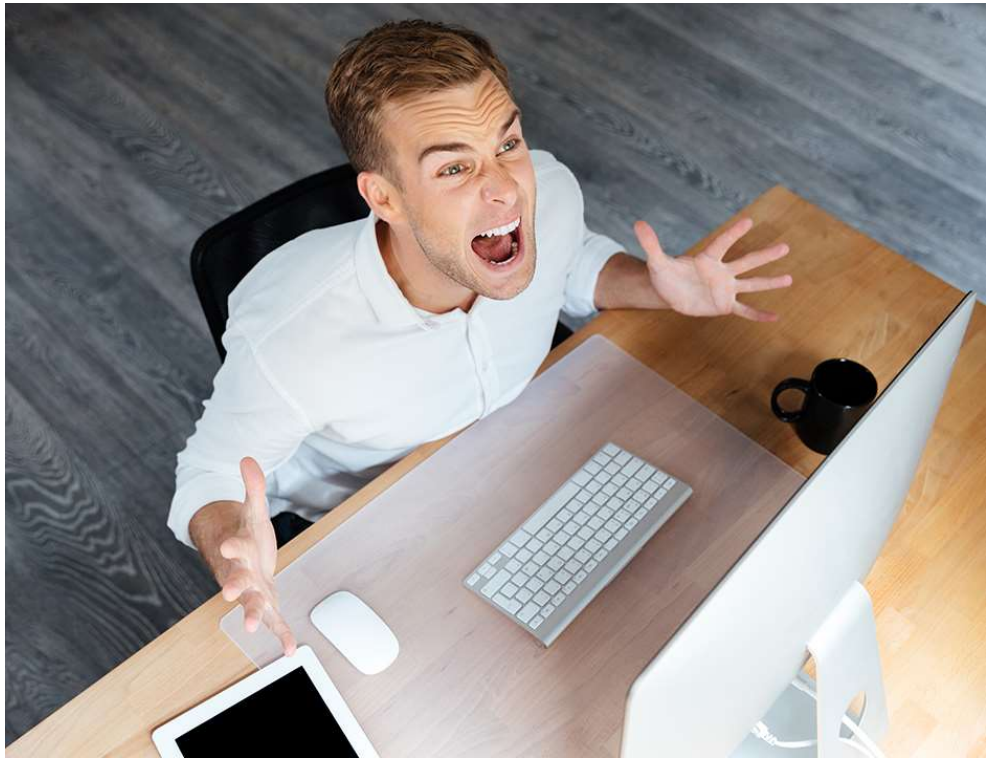
- Automated testing in Business Central involves using software to run tests automatically on the system to ensure that it functions as expected. This approach reduces manual testing, which can be time-consuming, error-prone, and costly.

- Automated testing in Business Central involves creating a set of test cases that simulate different scenarios and use cases. These test cases are then executed automatically and checks the output against the expected results. The results of each test are analyzed to identify any defects or issues.

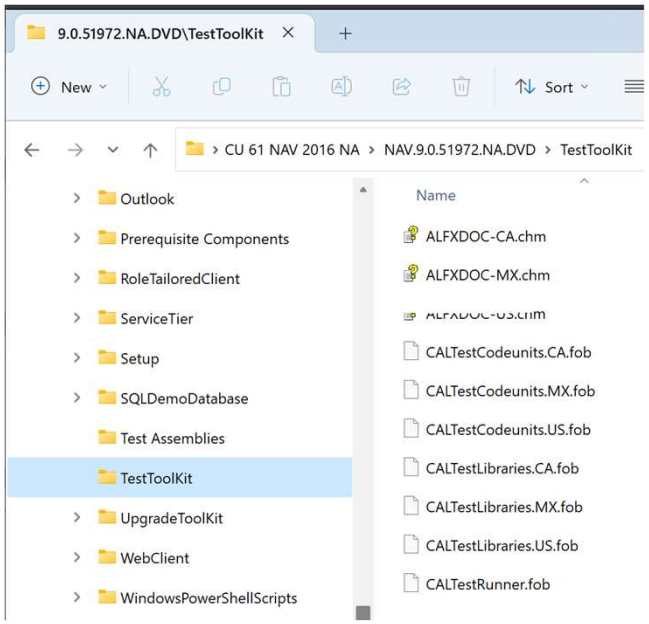


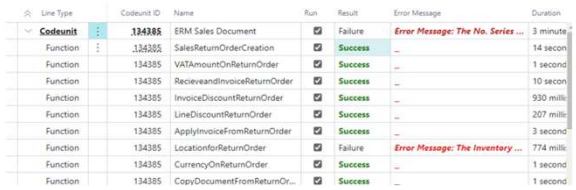
Why use Automated Testing

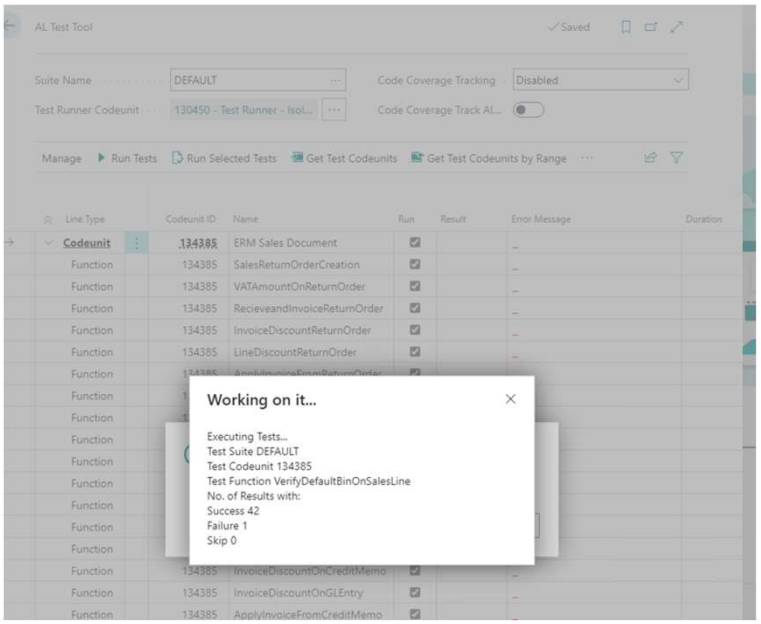
- Saves Time
- Improves Quality*
- Repeatable and Reliable
- Reduces Risk and Cost
- Less Disruption / Maximize Error Detection
- Latest Releases



Automated Testing in Business Central







Setting up an Environment

- Environment testing support and limitations

Business Central solution	Environment	Testing allowed	More details
Online	Production		Running tests isn't allowed because it might have an adverse effect on your business. Testing can incidentally invoke external systems, like CDS, PayPal, and web hook subscriptions. Invoking these systems may slow down the solution for other users or cause data corruption.
	Sandbox	✓	You can use a sandbox environment to run tests manually to verify functionality on an environment. Running a large number of tests or tests that take a long time (more than 15 minutes per test method) isn't allowed. It's recommended that you don't run tests more than one or two hours a day.
On-premises	Production	✓	For Business Central on-premises, running automated tests is only possible with a Partner license or a license that includes the Application Builder module. You can disable the ability to run tests by turning off **Enable Test Automation** (TestAutomationEnabled) on the Business Central Server instance. For more information, see Configuring Business Central Server - General Settings.
	Container-based development environment	✓	This setup should be the default environment for running large number of tests or setting up CI/CD gates. For more information, see Running a Container-Based Development Environment or Running Tests In Containers.

Source https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/developer/devenv-testing-application

Setting up an Environment

- Available on Microsoft Dynamics 365 Business Central installation “DVD”

Applications › testframework ›

Name	Date modified	Type	Size
performancetoolkit	3/3/2023 3:47 PM	File folder	
TestLibraries	2/3/2023 8:13 AM	File folder	
TestRunner	3/3/2023 3:47 PM	File folder	
teststabilitytools	2/3/2023 8:13 AM	File folder	

Setting up an Environment

- BCContainerHelper: NewBCContainer

```
$startTime = [DateTime]::Now
Write-Host -ForegroundColor Green "$startTime : New-BCContainer start"

$artifactUrl = Get-BCArtifactUrl -version 22 -country "us" -select Latest -type Sandbox
Write-Host -ForegroundColor Yellow ($artifactUrl)

New-BCContainer `
    -accept_eula `
    -accept_outdated `
    -alwaysPull `
    -artifactUrl $artifactUrl `
    -auth NavUserPassword `
    -containerName $containerName `
    -Credential $credential `
    -includeTestToolkit `
    -isolation hyperv `
    -updateHosts `

$timeSpend = [Math]::Round([DateTime]::Now.Subtract($startTime).Totalseconds)
$stopTime = [DateTime]::Now

Write-Host -ForegroundColor Green "$stopTime : New-BCContainer took $timespend seconds"
```

```
-includeTestToolkit
-includeTestLibrariesOnly
-includeTestFrameworkOnly
```

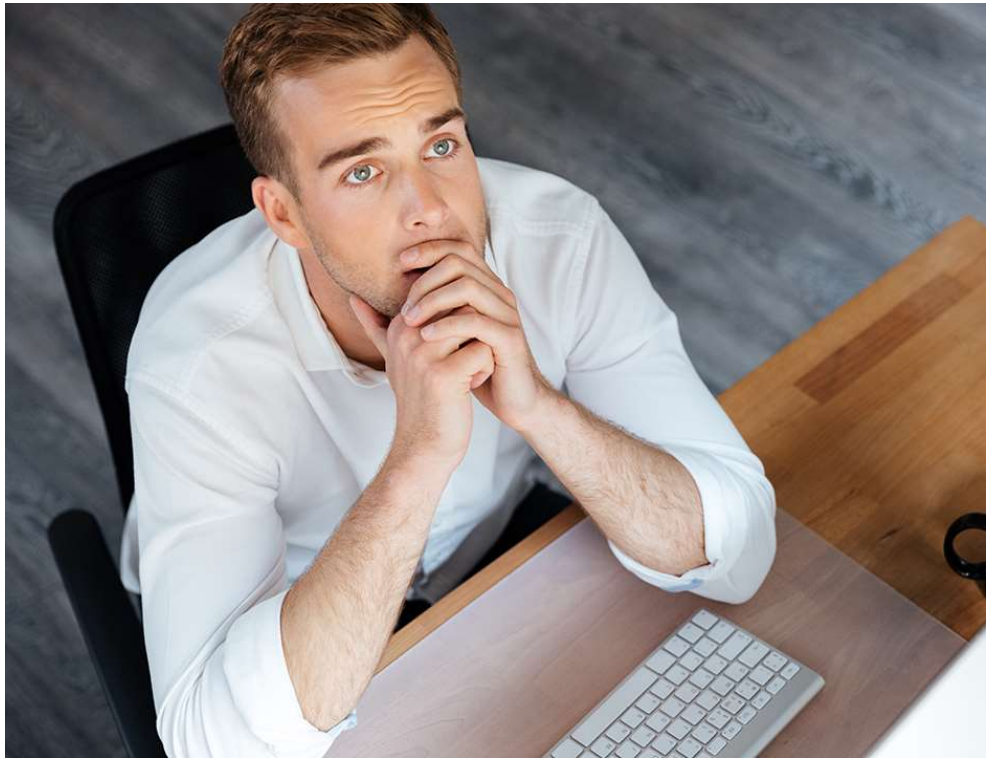
Setting up an Environment

- BCContainerHelper: ImportTestToolkitToBCContainer

```
##########################################################################################

$containerName = 'BC22-Test'

##########################################################################################

$startTime = [DateTime]::Now
Write-Host -ForegroundColor Green "$startTime : Import-TestToolkitToBCContainer start"

Import-TestToolkitToBCContainer -containerName $containerName

$timeSpend = [Math]::Round([DateTime]::Now.Subtract($startTime).Totalseconds)
$stopTime = [DateTime]::Now

Write-Host -ForegroundColor Green "$stopTime : Import-TestToolkitToBCContainer took $timeSpend seconds"
```

Writing Tests

- Where do you start?
  - Testing Plan
  - Testing Scenarios
  - Code Coverage
  - Test Driven Development



Writing Tests

- What do we test?
  - Test Pages
  - Performance Toolkit Extension
  - Test Codeunits
  - UI Handlers

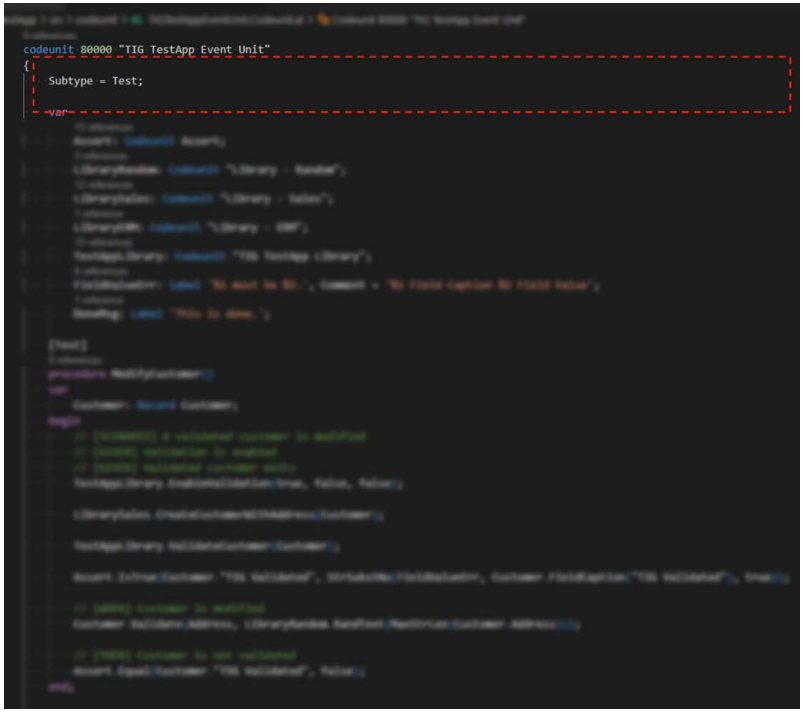
Writing Tests

- TestApp Dependencies


```
"dependencies": [
  {
    "id": "a3c9c3aa-6c42-4cba-aaea-33d9a155f1b1",
    "name": "MainApp",
    "publisher": "Default publisher",
    "version": "22.0.0.0"
  },
  {
    "id": "dd0be2ea-f733-4d65-bb34-a28f4624fb14",
    "publisher": "Microsoft",
    "name": "Library Assert",
    "version": "22.0.0.0"
  },
  {
    "id": "e7320ebb-08b3-4406-b1ec-b4927d3e280b",
    "publisher": "Microsoft",
    "name": "Any",
    "version": "22.0.0.0"
  },
  {
    "id": "5d86850b-0d76-4eca-bd7b-951ad998e997",
    "publisher": "Microsoft",
    "name": "Tests-TestLibraries",
    "version": "22.0.0.0"
  },
  {
    "id": "9856ae4f-d1a7-46ef-89bb-6ef056398228",
    "publisher": "Microsoft",
    "name": "System Application Test Library",
    "version": "22.0.0.0"
  }
],
```

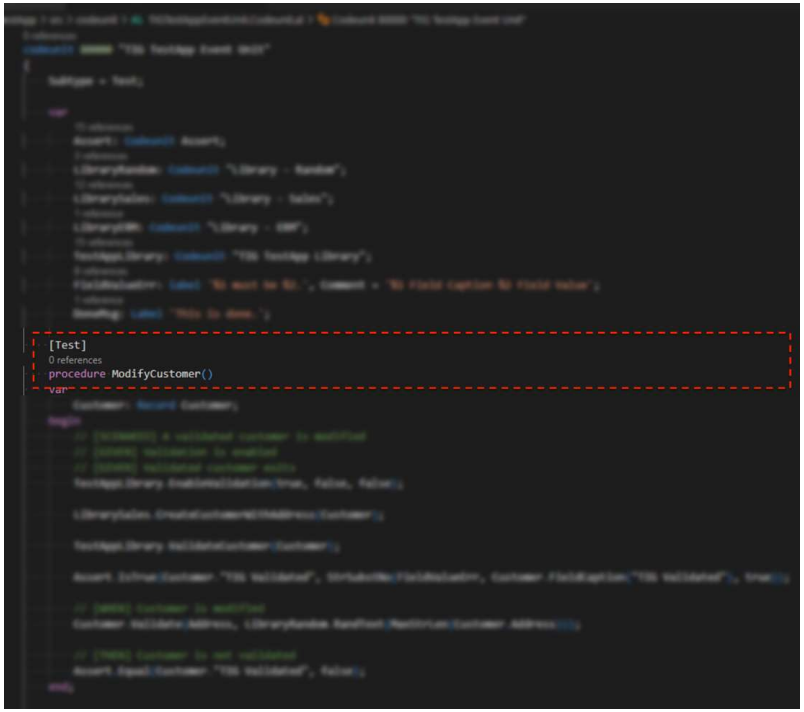
Writing Tests

- Test Codeunits: Subtype Property



Writing Tests

- Test Codeunits: Test Attribute on Methods



Writing Tests

- Test Codeunits: Handler Methods

Method type	Purpose	Signature
MessageHandler	Handles Message statements.	<Function name>(<Message>: Text[1024])Error! Filename not specified.
ConfirmHandler	Handles Confirm statements.	<Function name>(<Question>: Text[1024]; var <Reply>: Boolean)
StrMenuHandler	Handles StrMenu statements.	<Function name>(<Options: Text[1024]; var <Choice>: Integer; <Instruction>: Text[1024])
ReportHandler	Handles specific reports. If you create a ReportHandler method, then that method replaces all code for running the report, including the request page, and a RequestPageHandler isn't called. Only create a RequestPageHandler if you aren't using a ReportHandler.	<Function name>(var <Report>: Report <report id>)

Source https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/developer/devenv-creating-handler-methods

Writing Tests

- Gherkin Scenario

```
    // [SCENARIO] A description of the behavior you're going to test
    // [GIVEN] Beginning state of the scenario; what is needed to run the test
    // [WHEN] A specific action the user takes; Something Happens
    // [THEN] A testable outcome; usually caused by the [WHEN] Action; Do your test
```

Writing Tests

- Assert Library

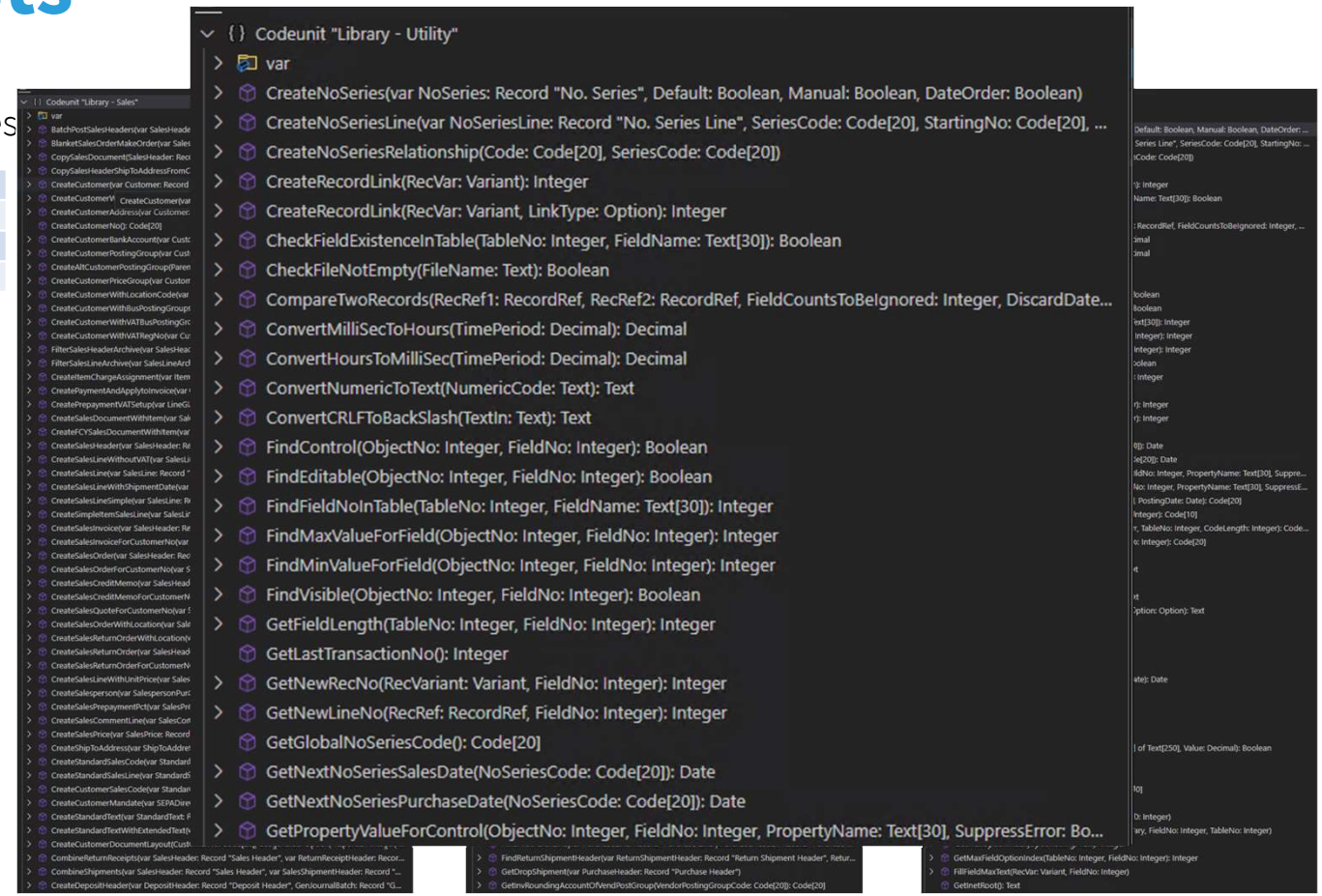
```
/// <summary>
/// This module provides functions for easy verification of expected values and error handling in test code.
/// </summary>
codeunit 130002 "Library Assert"
{
```

```
{} Codeunit "Library Assert"
  OnRun()
  var
  IsTrue(Condition: Boolean; Msg: Text)
  IsFalse(Condition: Boolean; Msg: Text)
  AreEqual(ExpectedVariant: Variant; ActualVariant: Variant; Msg: Text)
  AreEqual(Expected: Dictionary of [Text, Text]; Actual: Dictionary of [Text, Text])
  AreEqualDateTime(Expected: DateTime; Actual: DateTime; Msg: Text)
  AreNotEqual(ExpectedVariant: Variant; ActualVariant: Variant; Msg: Text)
  AreNearlyEqual(Expected: Decimal; Actual: Decimal; Delta: Decimal; Msg: Text)
  AreNotNearlyEqual(Expected: Decimal; Actual: Decimal; Delta: Decimal; Msg: Text)
  Fail(Msg: Text)
  RecordIsEmpty(RecVariant: Variant)
  RecordIsNotEmpty(RecVariant: Variant)
  TableIsEmpty(TableNo: Integer)
  TableIsNotEmpty(TableNo: Integer)
  RecRefIsEmpty(var RecordRef: RecordRef)
  RecRefIsNotEmpty(var RecordRef: RecordRef)
  RecordCount(RecVariant: Variant; ExpectedCount: Integer)
  KnownFailure(Expected: Text; WorkItemNo: Integer)
  ExpectedError(Expected: Text)
  ExpectedErrorCode(Expected: Text)
  ExpectedMessage(Expected: Text; Actual: Text)
  AssertRecordNotFound()
  AssertRecordAlreadyExists()
  AssertNothingInsideFilter()
  AssertNoFilter()
  AssertPrimRecordNotFound()
  TypeOf(ValueVariant: Variant) : Integer
  TypeNameOf(ValueVariant: Variant) : Text
  UnsupportedTypeName(ValueVariant: Variant) : Text
  Equal(LeftVariant: Variant; RightVariant: Variant) : Boolean
  IsNumber(ValueVariant: Variant) : Boolean
  EqualNumbers(Left: Decimal; Right: Decimal) : Boolean
  VerifyFailure(ErrorCodeExpected: Text; FailureText: Text)
```

Writing Tests

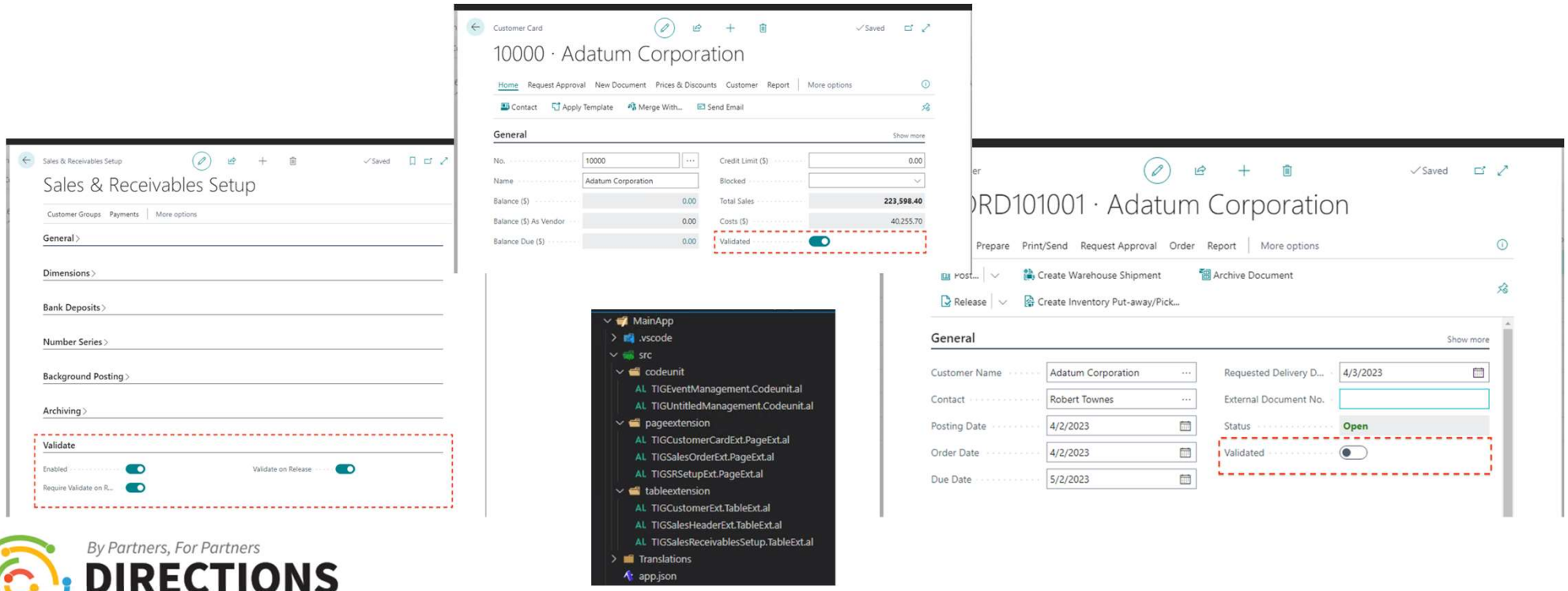
Microsoft_Tests-TestLibraries

CU	130440	Library - Random
CU	130509	Library – Sales
CU	130512	Library - Purchase
CU	131000	Library – Utility



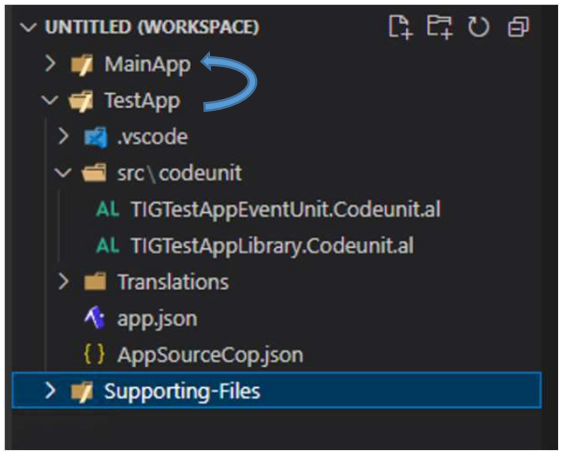
Writing Tests

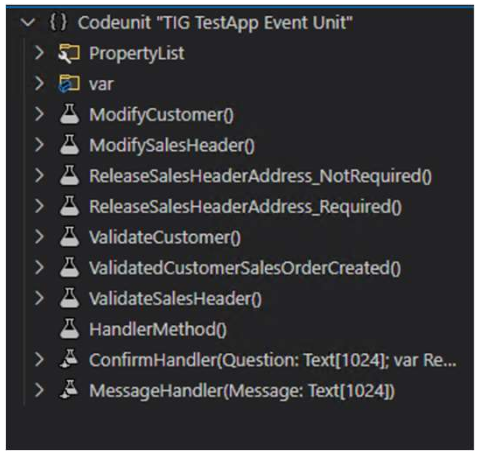
- Example App: MainApp



Writing Tests

- Example App: TestApp





Writing Tests

- Example App: TestApp Methods

```
codeunit 80000 "TIG TestApp Event Unit"
{
  Subtype = Test;
  var
      15 references
      Assert: Codeunit Assert;
      3 references
      LibraryRandom: Codeunit "Library - Random";
      12 references
      LibrarySales: Codeunit "Library - Sales";
      1 reference
      LibraryERM: Codeunit "Library - ERM";
      15 references
      TestAppLibrary: Codeunit "TIG TestApp Library";
      8 references
      FieldValueErr: label '%1 must be %2.', Comment = '%1 Field Caption %2 Field Value';
      1 reference
      DoneMsg: Label 'This is done.';
  [Test]
  0 references
  procedure ModifyCustomer()
  var
      Customer: Record Customer;
  begin
      // [SCENARIO] A validated customer is modified
      // [GIVEN] Validation is enabled
      // [GIVEN] Validated customer exits
      TestAppLibrary.EnableValidation(true, false, false);

      LibrarySales.CreateCustomerWithAddress(Customer);

      TestAppLibrary.ValidateCustomer(Customer);

      Assert.IsTrue(Customer."TIG Validated", StrSubstNo(FieldValueErr, Customer.FieldCaption("TIG Validated"), true));

      // [WHEN] Customer is modified
      Customer.Validate(Address, LibraryRandom.RandText(MaxStrLen(Customer.Address)));

      // [THEN] Customer is not validated
      Assert.Equal(Customer."TIG Validated", false);
  end;
```

```
[Test]
0 references
procedure ModifySalesHeader()
var
    Customer: Record Customer;
    SalesHeader: Record "Sales Header";
    CustomerNo: Code[20];
begin
    // [SCENARIO] A validated sales header is modified
    // [GIVEN] Validation is enabled
    // [GIVEN] Validated sales header exists
    TestAppLibrary.EnableValidation(true, false, false);

    LibrarySales.CreateCustomerWithAddress(Customer);
    CustomerNo := Customer."No.";

    LibrarySales.CreateSalesHeader(SalesHeader, "Sales Document Type"::Order, CustomerNo);

    TestAppLibrary.ValidateSalesHeader(SalesHeader);

    Assert.IsTrue(SalesHeader."TIG Validated", StrSubstNo(FieldValueErr, SalesHeader.FieldCaption("TIG Validated"), true));

    // [WHEN] Sales header is modified
    SalesHeader.Validate("Sell-to Address", LibraryRandom.RandText(MaxStrLen(SalesHeader."Sell-to Address")));

    // [THEN] Sales header is not validated
    Assert.Equal(SalesHeader."TIG Validated", false);
end;

[Test]
0 references
procedure ReleaseSalesHeaderAddress_NotRequired()
var
    SalesHeader: Record "Sales Header";

begin
    // [SCENARIO] User releases sales order that is not validated and validation on release is enabled
    // [GIVEN] Validation is enabled and required on release
    // [GIVEN] Sales order exists and is not validated
    TestAppLibrary.EnableValidation(true, false, false);

    LibrarySales.CreateSalesOrder(SalesHeader);
    SalesHeader.Modify();

    Assert.IsFalse(SalesHeader."TIG Validated", StrSubstNo(FieldValueErr, SalesHeader.FieldCaption("TIG Validated"), false));

    // [WHEN] Sales Header released
    TestAppLibrary.ReleaseSalesDocumentCheckSkip(SalesHeader, false, true);

    // [THEN] Sales Header is relesed
    Assert.Equal(SalesHeader.Status, "Sales Document Status"::Released);
end;
```

Writing Tests

- Example App: TestApp Methods

```
[Test]
0 references
procedure ReleaseSalesHeaderAddress_Required()
var
    Customer: Record Customer;
    SalesHeader: Record "Sales Header";
    SalesLine: Record "Sales Line";
    CustomerNo: Code[20];
    GLAccNo: Code[20];

begin
    // [SCENARIO] User releases sales order that is not validated and validation on release is enabled
    // [GIVEN] Validation is enabled and required on release
    // [GIVEN] Sales order exists and is not validated
    TestAppLibrary.EnableValidation(true, true, true);

    LibrarySales.CreateCustomerWithAddress(Customer);
    CustomerNo := Customer."No.";

    GLAccNo := LibraryERM.CreateGLAccountWithSalesSetup();

    LibrarySales.CreateSalesHeader(SalesHeader, "Sales Document Type"::Order, CustomerNo);
    SalesHeader.Modify();

    LibrarySales.CreateSalesLine(SalesLine, SalesHeader, "Sales Line Type"::"G/L Account", GLAccNo, LibraryRandom.RandInt(5));
    SalesLine.Modify();

    Assert.IsFalse(SalesHeader."TIG Validated", StrSubstNo(FieldValueErr, SalesHeader.FieldCaption("TIG Validated"), false));

    // [WHEN] Sales Header released
    TestAppLibrary.ReleaseSalesDocumentCheckSkip(SalesHeader, false, true);

    // [THEN] Sales Header is not relesed
    Assert.Equal(SalesHeader.Status, "Sales Document Status"::Open);
end;
```

Writing Tests

- Example App: TestApp Handlers

```
codeunit 50100 "TIG Untitled Management"
    1 reference
    procedure ConfirmMessage()
    var
        ConfirmMsg: Label 'Are you sure that you want to do this?';
        DoneMsg: Label 'This is done.';
        NotDoneMsg: Label 'This is not done.';

    begin
        if Confirm(ConfirmMsg, false) then
            Message(DoneMsg)
        else
            Message(NotDoneMsg);
    end;
```

```
[Test]
[HandlerFunctions('MessageHandler,ConfirmHandler')]
0 references
procedure HandlerMethod()
begin
    // [SCENARIO] Demonstrate handler methonds
    // [GIVEN] Handlers assigned
    // [WHEN] Test performance with confirm and messaege
    TestAppLibrary.ConfirmHandler();
    // [THEN] No Assert
end;

[ConfirmHandler]
0 references
procedure ConfirmHandler(Question: Text[1024]; var Reply: Boolean);
begin
    Reply := true;
end;

[MessageHandler]
0 references
procedure MessageHandler(Message: Text[1024])
begin
    Assert.IsTrue(StrPos(Message, DoneMsg) > 0, Message);
end;
```

```
codeunit 80001 "TIG TestApp Library"
    1 reference
    procedure ConfirmHandler()
    var
        UntitledManagement: Codeunit "TIG Untitled Management";
    begin
        UntitledManagement.ConfirmMessage();
    end;
}
```

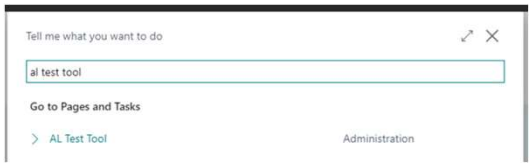
Writing Tests

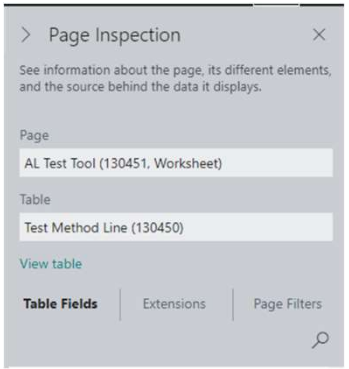
- Example App: TestApp Library

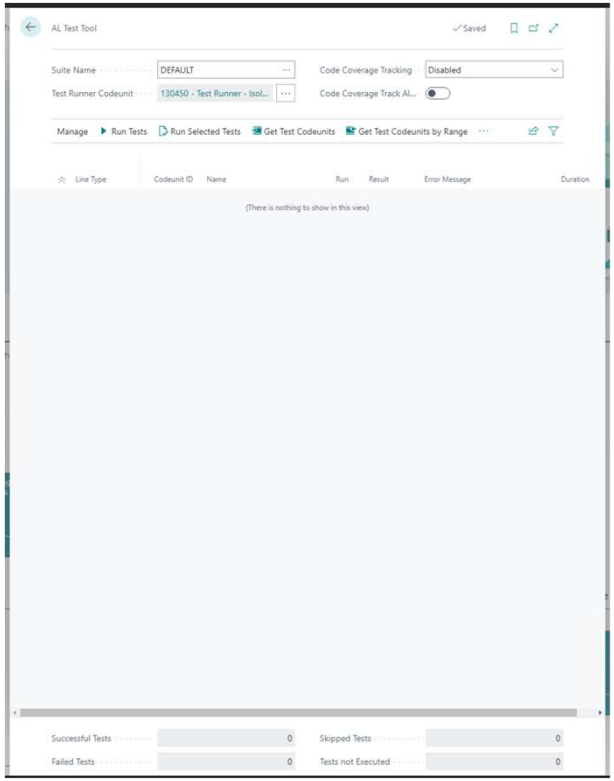
```
1 reference
codeunit 80001 "TIG TestApp Library"
{
    3 references
    procedure ValidateCustomer(var Customer: Record Customer)
    var
        UntitledManagement: Codeunit "TIG Untitled Management";
    begin
        UntitledManagement.ValidateCustomer(Customer, true);
    end;

    2 references
    procedure ValidateSalesHeader(var SalesHeader: Record "Sales Header")
    var
        UntitledManagement: Codeunit "TIG Untitled Management";
    begin
        UntitledManagement.ValidateSalesHeader(SalesHeader, true);
    end;

    7 references
    procedure EnableValidation(Enabled: Boolean; ReqValidateonRelease: Boolean; ValidateonRelease: Boolean)
    var
        SalesReceivablesSetup: Record "Sales & Receivables Setup";
    begin

        if not SalesReceivablesSetup.FindFirst() then
            SalesReceivablesSetup.Init();

        SalesReceivablesSetup."TIG Enabled" := Enabled;
        SalesReceivablesSetup."TIG Req Validate on Release" := ReqValidateonRelease;
        SalesReceivablesSetup."TIG Validate on Release" := ValidateonRelease;
        if not SalesReceivablesSetup.insert() then
            SalesReceivablesSetup.Modify();
    end;

    2 references
    procedure ReleaseSalesDocumentCheckSkip(var SalesHeader: Record "Sales Header"; SkipCheckReleaseRestrictions: Boolean; Commit: Boolean): Boolean
    var
        ReleaseSalesDocument: Codeunit "Release Sales Document";
    begin
        if SkipCheckReleaseRestrictions then
            ReleaseSalesDocument.SetSkipCheckReleaseRestrictions();

        if Commit then
            Commit();

        exit(ReleaseSalesDocument.Run(SalesHeader));
    end;

    1 reference
    procedure ConfirmHandler()
    var
        UntitledManagement: Codeunit "TIG Untitled Management";
    begin
        UntitledManagement.ConfirmMessage();
    end;
}
```

Running Tests

- AL Test Tool







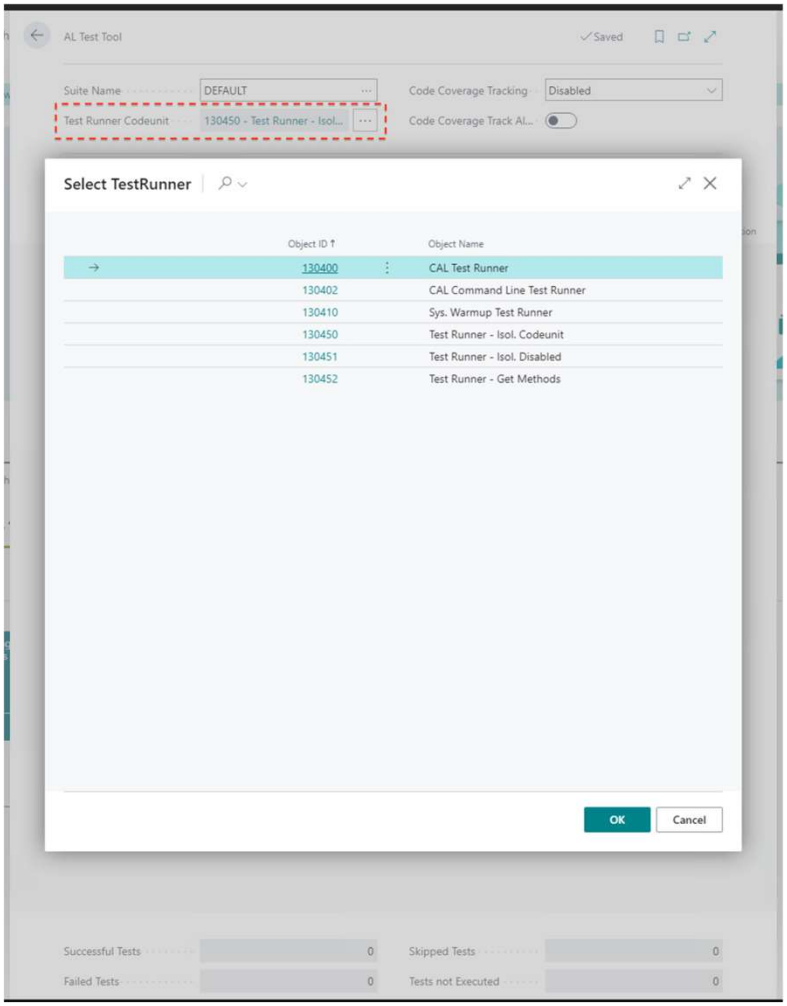
Running Tests

- Test Runner Codeunit

TestIsolation Property

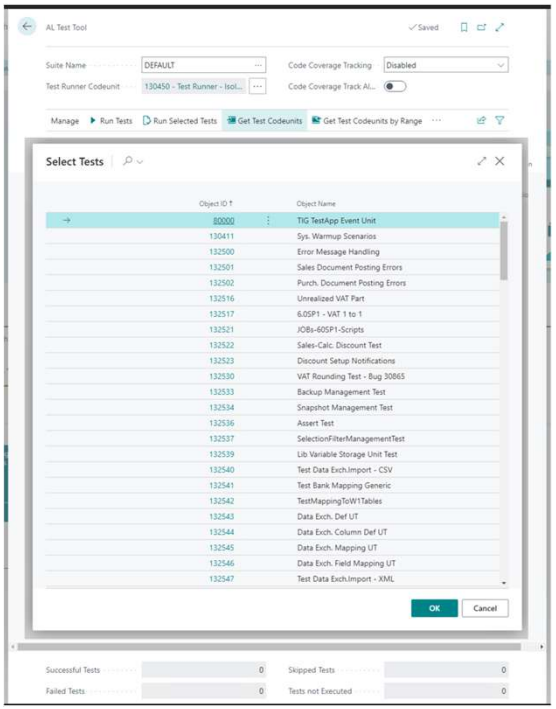
Value	Description
Disabled	Do not roll back any changes to the database. Tests are not isolated from each other. This is the default value.
Codeunit	Roll back all changes to the database after each test codeunit executes.
Function	Roll back all changes to the database after each test method executes.

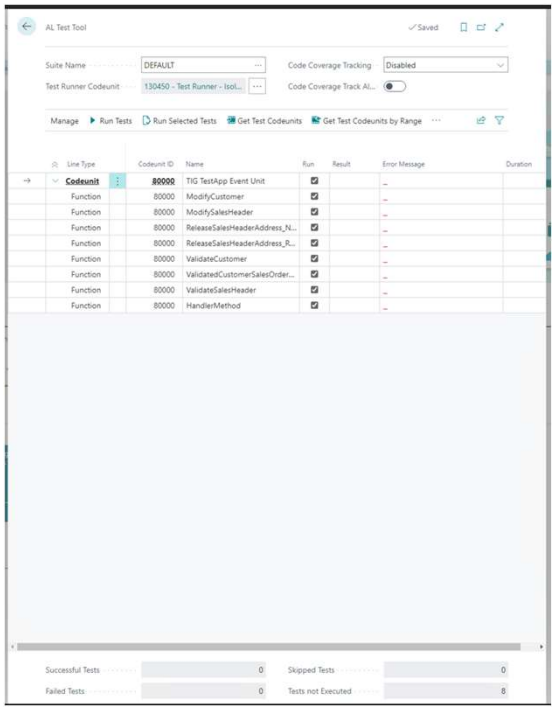
Source https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/developer/properties/devenv-testisolation-property



Running Tests

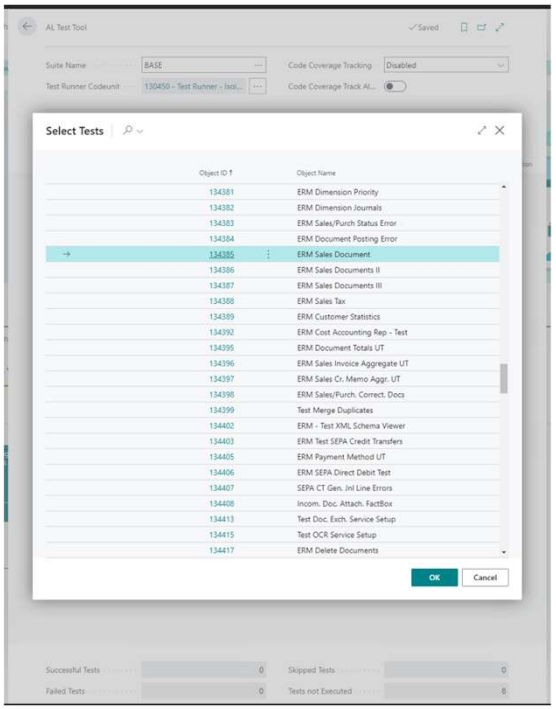
- Test Codeunits

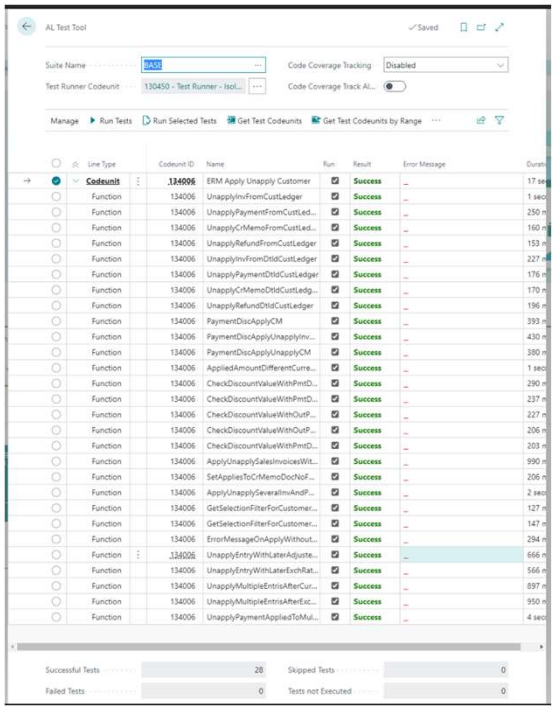




Running Tests

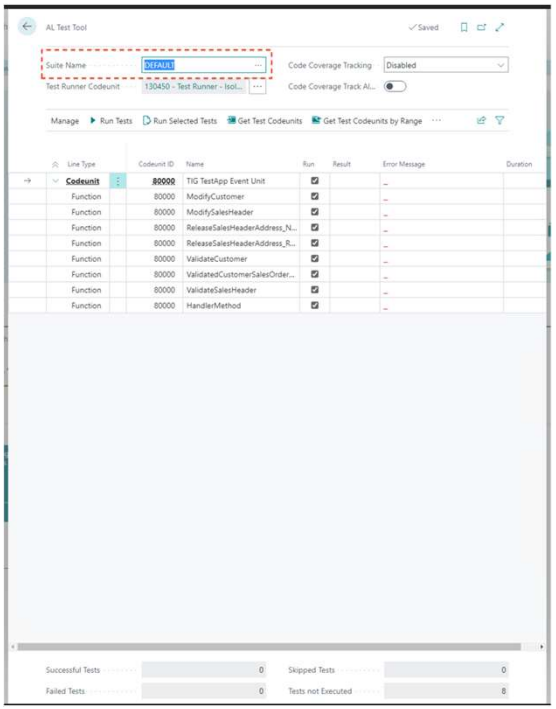
- Base Tests

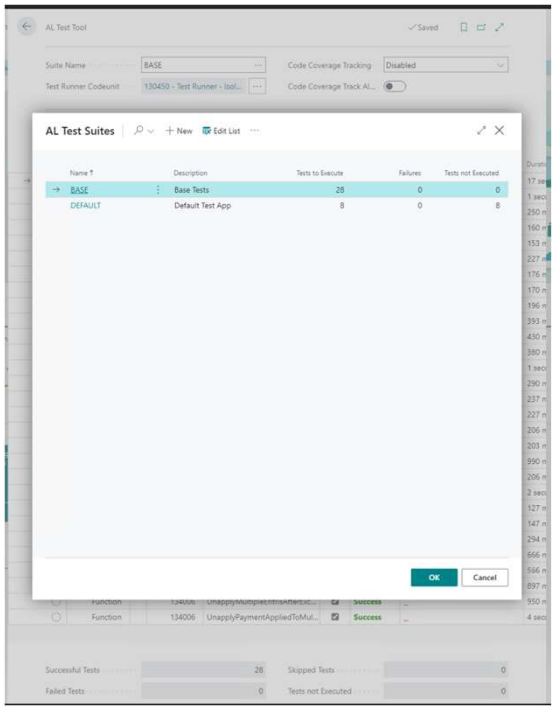




Running Tests

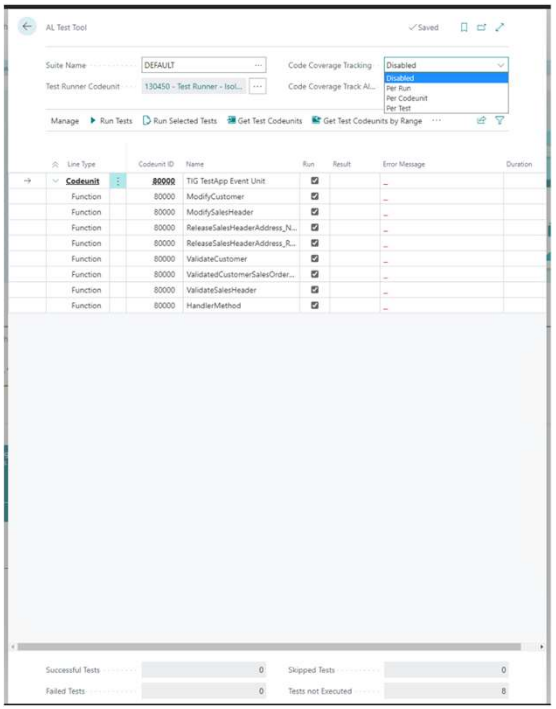
- Test Suite

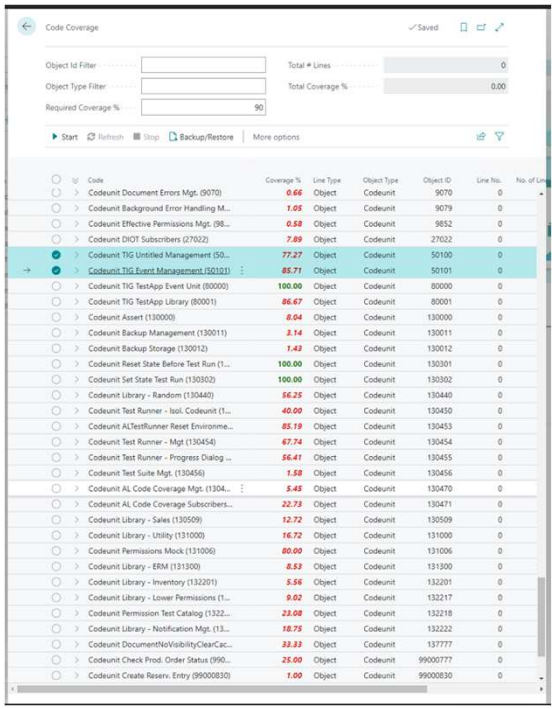




Running Tests

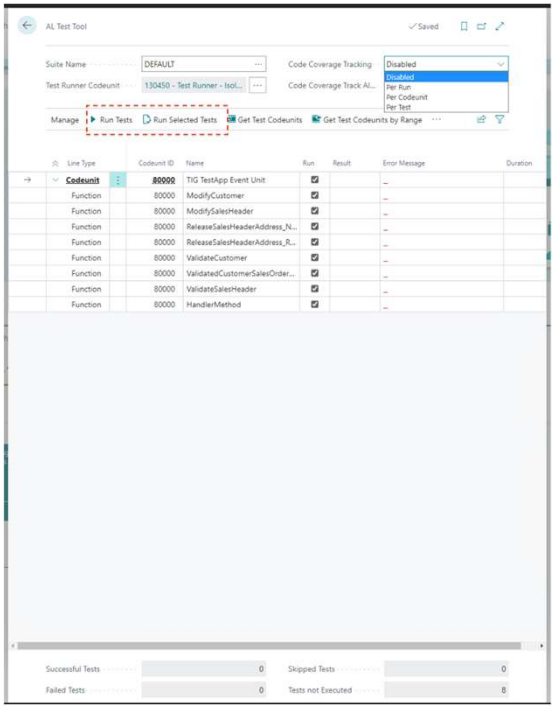
- Code Coverage

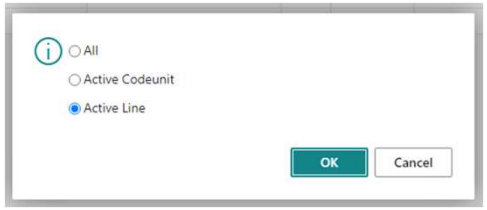




Running Tests

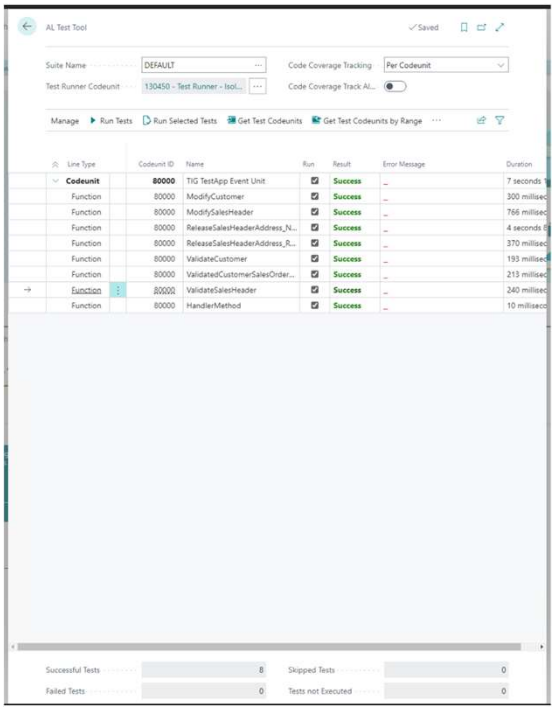
- Running Tests

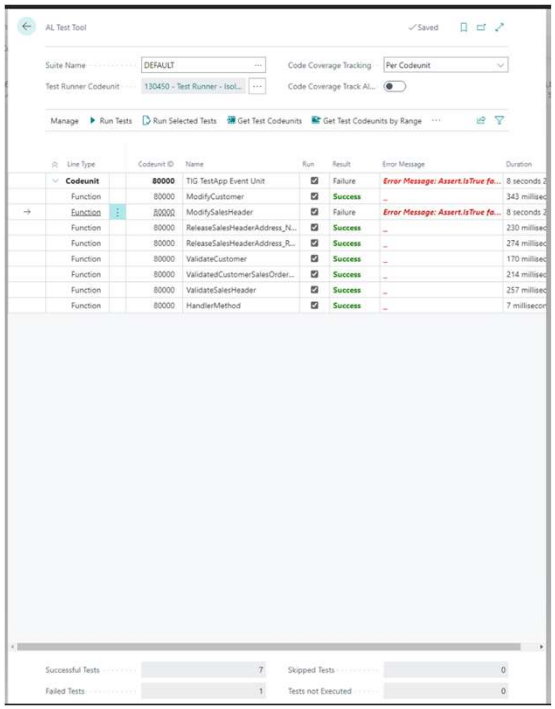




Running Tests

- Completed Tests





Running Tests

- BCContainerHelper: Get-TestsFromBCContainer

Get-TestsFromBCContainer-1.ps1

Supporting-Files > Powershell > Get-TestsFromBCContainer-1.ps1 > ...

```
1  ##########################################################################################
2
3  $containerName = 'BC22-Test'
4  $testSuite = 'DEFAULT'
5  $userName = 'admin'
6  $password = 'password'
7
8  ##########################################################################################
9
10 $scriptTitle = 'Get-TestsFromBCContainer Pipeline'
11
12 $startTime = [DateTime]::Now
13 Write-Host -ForegroundColor Green "$($startTime) : $($scriptTitle) start"
14
15 $securePassword = ConvertTo-SecureString $password -AsPlainText -Force
16 $credential = New-Object System.Management.Automation.PSCredential ($userName,$securePassword)
17
18 Get-TestsFromBCContainer `
19     -containerName $containerName `
20     -Credential $credential `
21     -testSuite $testSuite `
22     -ignoreGroups
23
24 $stopTime = [DateTime]::Now
25 $timeSpend = ($stopTime -$startTime).TotalSeconds
26
27 Write-Host -ForegroundColor Green "$($stopTime) : $($scriptTitle) took $($timeSpend) seconds"
```

```
04/10/2023 16:32:28 : Get-TestsFromBCContainer Pipeline start

04/10/2023 16:32:34 : Get-TestsFromBCContainer Pipeline took 6.3243889 seconds
Tests                                                                                                  Name                  Id
-----                                                                                                  ----                  --
{ModifyCustomer, ModifySalesHeader, ReleaseSalesHeaderAddress_NotRequired, ReleaseSalesHeaderAddress_Required...} TIG TestApp Event Unit 80000
```

Running Tests

- BCContainerHelper: Get-TestsFromBCContainer

Get-TestsFromBCContainer-2.ps1

Supporting-Files > Powershell > Get-TestsFromBCContainer-2.ps1 > ...

```
1  ##########################################################################################
2
3  $containerName = 'BC22-Test'
4  $testSuite = 'DEFAULT'
5  $userName = 'admin'
6  $password = 'password'
7
8  ##########################################################################################
9
10 $scriptTitle = 'Get-TestsFromBCContainer'
11
12 $startTime = [DateTime]::Now
13 Write-Host -ForegroundColor Green "$($startTime) : $($scriptTitle) start"
14
15 $securePassword = ConvertTo-SecureString $password -AsPlainText -Force
16 $credential = New-Object System.Management.Automation.PSCredential ($userName,$securePassword)
17
18 $tests = Get-TestsFromBCContainer `
19     -containerName $containerName `
20     -Credential $credential `
21     -testSuite $testSuite `
22     -ignoreGroups
23
24 foreach( $test in $tests) {
25     Write-Host -ForegroundColor Yellow $tests.Id $tests.Name
26     Write-Output $test.Tests
27 }
28
29
30 $stopTime = [DateTime]::Now
31 $timeSpend = ($stopTime -$startTime).TotalSeconds
32
33 Write-Host -ForegroundColor Green "$($stopTime) : $($scriptTitle) took $($timeSpend) seconds"
```

```
04/10/2023 16:33:57 : Get-TestsFromBCContainer start
80000 TIG TestApp Event Unit
ModifyCustomer
ModifySalesHeader
ReleaseSalesHeaderAddress_NotRequired
ReleaseSalesHeaderAddress_Required
ValidateCustomer
ValidatedCustomerSalesOrderCreated
ValidateSalesHeader
HandlerMethod
04/10/2023 16:34:03 : Get-TestsFromBCContainer took 6.7750794 seconds
```

Running Tests

- BCContainerHelper: Run-TestsInBCContainer

Run-TestsInBCContainer.ps1

Supporting-Files > Powershell > Run-TestsInBCContainer.ps1 > ...

```
1  ##########################################################################################
2
3  $containerName = 'BC22-Test'
4  $userName = 'admin'
5  $password = 'password'
6  $testSuite = 'DEFAULT'
7  $xunitResultsFile = "C:\ProgramData\BcContainerHelper\Extensions\$($containerName)\results.xml"
8
9  ##########################################################################################
10
11 $scriptTitle = 'Run-TestsInBCContainer'
12
13 $startTime = [DateTime]::Now
14 Write-Host -ForegroundColor Green "$($startTime) : $($scriptTitle) start"
15
16 $securePassword = ConvertTo-SecureString $password -AsPlainText -Force
17 $Credential = New-Object System.Management.Automation.PSCredential ($userName,$securePassword)
18
19 Run-TestsInBCContainer `
20     -containerName $containerName `
21     -credential $credential `
22     -testSuite $testSuite `
23     -detailed `
24     -AppendToXUnitResultFile:($true) `
25     -XUnitResultFileName $xunitResultsFile
26
27 $stopTime = [DateTime]::Now
28 $timeSpend = ($stopTime -$startTime).TotalSeconds
29
30 Write-Host -ForegroundColor Green "$($stopTime) : $($scriptTitle) took $timeS
```

```
04/10/2023 16:35:02 : Run-TestsInBCContainer start

Connecting to http://localhost:80/BC/cs?tenant=default
  Codeunit 80000 TIG TestApp Event Unit Success (7.79 seconds)
    Testfunction ModifyCustomer Success (1.94 seconds)
    Testfunction ModifySalesHeader Success (1.397 seconds)
    Testfunction ReleaseSalesHeaderAddress_NotRequired Success (1.543 seconds)
    Testfunction ReleaseSalesHeaderAddress_Required Success (0.333 seconds)
    Testfunction ValidateCustomer Success (0.22 seconds)
    Testfunction ValidatedCustomerSalesOrderCreated Success (0.263 seconds)
    Testfunction ValidateSalesHeader Success (0.196 seconds)
    Testfunction HandlerMethod Success (0.007 seconds)
04/10/2023 16:35:22 : Run-TestsInBCContainer took 19.1651415 seconds
```

Running Tests

- BCContainerHelper: Run-TestsInBCContainer

Get-TestsFromBCContainer-pipline.ps1

Supporting-Files > Powershell > Get-TestsFromBCContainer-pipline.ps1 > ...

```
##########################################################################################

$containerName = 'BC22-Test'
$userName = 'admin'
$password = 'password'
$testSuite = 'DEFAULT'
$xunitResultsFile = "C:\ProgramData\BcContainerHelper\Extensions\$($containerName)\results.

##########################################################################################

$scriptTitle = 'Get-TestsFromBCContainer Pipeline'

$startTime = [DateTime]::Now
Write-Host -ForegroundColor Green "$($startTime) : $($scriptTitle) start"

$securePassword = ConvertTo-SecureString $Password -AsPlainText -Force
$credential = New-Object System.Management.Automation.PSCredential ($userName,$securePassword)

$first = $true
$tests = Get-TestsFromBCContainer `
    -containerName $containerName `
    -Credential $Credential `
    -testSuite $testSuite `
    -ignoreGroups

$tests | ForEach-Object {
    Run-TestsInBCContainer `
        -containerName $containerName `
        -credential $credential `
        -XUnitResultFileName $xunitResultsFile `
        -AppendToXUnitResultFile:(!$first) `
        -testCodeunit $_.Id
    $first = $false
}

$stopTime = [DateTime]::Now
$timeSpend = ($stopTime -$startTime).TotalSeconds

Write-Host -ForegroundColor Green "$($stopTime) : $($scriptTitle) took $($timeSpend) seconds"
```

```
04/10/2023 16:41:53 : Get-TestsFromBCContainer Pipeline start

Connecting to http://localhost:80/BC/cs?tenant=default
  Codeunit 80000 TIG TestApp Event Unit Success (9.447 seconds)
04/10/2023 16:42:25 : Get-TestsFromBCContainer Pipeline took 32.6839124 seconds

PS C:\Windows\system32>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<assemblies>
  <assembly name="80000 TIG TestApp Event Unit" test-framework="PS Test Runner" run-date="2023-04-10" run-time="17:49:08" total="8" passed="8" failed="0" skipped="0" time="7.777">
    <collection name="TIG TestApp Event Unit" total="8" passed="8" failed="0" skipped="0" time="7.777" Skipped="0">
      <test name="TIG TestApp Event Unit:ModifyCustomer" method="ModifyCustomer" time="2.627" result="Pass" />
      <test name="TIG TestApp Event Unit:ModifySalesHeader" method="ModifySalesHeader" time="1.983" result="Pass" />
      <test name="TIG TestApp Event Unit:ReleaseSalesHeaderAddress_NotRequired" method="ReleaseSalesHeaderAddress_NotRequired" time="2.01" result="Pass" />
      <test name="TIG TestApp Event Unit:ReleaseSalesHeaderAddress_Required" method="ReleaseSalesHeaderAddress_Required" time="0.4" result="Pass" />
      <test name="TIG TestApp Event Unit:ValidateCustomer" method="ValidateCustomer" time="0.256" result="Pass" />
      <test name="TIG TestApp Event Unit:ValidatedCustomerSalesOrderCreated" method="ValidatedCustomerSalesOrderCreated" time="0.24" result="Pass" />
      <test name="TIG TestApp Event Unit:ValidateSalesHeader" method="ValidateSalesHeader" time="0.247" result="Pass" />
      <test name="TIG TestApp Event Unit:HandlerMethod" method="HandlerMethod" time="0.014" result="Pass" />
    </collection>
  </assembly>
</assemblies>
```

Running Tests

- BCContainerHelper: Run-AlPipeline

Run-AlPipeline.ps1

Supporting-Files > Powershell > Run-AlPipeline.ps1 > ...

```
1  ##########################################################################################
2  # Variables
3
4  $baseFolder = "C:\Temp\Introduction-to-Test-Automation-in-Business-Central\"
5  $userName = 'admin'
6  $password = 'password'
7  $pipelineName = "DirectionsNA"
8  $appVersion = '22.0'
9  $affix = 'TIG'
10 $appFolders = "MainApp"
11 $testFolders = "TestApp"
12 $rulesetFile = Join-Path -Path $($baseFolder) -ChildPath $("Supporting-Files\untiltled.ruleset.json")
13 $outputFolder = "Pipeline"
14 $testResultsFile = "TestResults.xml"
15 $PackagesFolder = ".alpackages"
16 $outputFolder = "output"
17 $outputFolder = Join-Path -Path $baseFolder -ChildPath $outputFolder
18 $testResultsFile = Join-Path -Path $($baseFolder) -ChildPath $("TestResults.xml")
19
20 $securePassword = ConvertTo-SecureString $password -AsPlainText -Force
21 $credential = New-Object System.Management.Automation.PSCredential ($userName, $securePassword)
22
23 ##########################################################################################
24
```

Run-AlPipeline.ps1

Supporting-Files > Powershell > Run-AlPipeline.ps1 > ...

```
22
23 ##########################################################################################
24
25
26 $scriptTitle = 'Run-AlPipeline'
27
28 $startTime = [DateTime]::Now
29 Write-Host -ForegroundColor Green "$($startTime) : $($scriptTitle) start"
30
31 Run-AlPipeline `
32     -pipelineName $pipelineName `
33     -baseFolder $baseFolder `
34     -appFolders $appFolders `
35     -testFolders $testFolders `
36     -installTestFramework `
37     -installTestLibraries `
38     -enablePerTenantExtensionCop `
39     -enableUICop `
40     -enableCodeCop `
41     -enableAppSourceCop `
42     -appVersion $appVersion `
43     -Credential $credential `
44     -CreateRuntimePackages `
45     -AppSourceCopMandatoryAffixes $affix `
46     -AppSourceCopSupportedCountries "US" `
47     -rulesetFile $rulesetFile `
48     -outputFolder $outputFolder `
49     -testResultsFile $testResultsFile `
50     -packagesFolder $PackagesFolder
51
52
53 $stopTime = [DateTime]::Now
54 $timeSpend = ($stopTime -$startTime).TotalSeconds
55
56 Write-Host -ForegroundColor Green "$($stopTime) : $($scriptTitle) took $($timeSpend) seconds"
```

Running Tests

- BCContainerHelper: Run-AlPipeline

```
duction-to-Test-Automation-in-Business-Central\Supporting-Files\Powershell\Run-AlPipeline.ps1'
04/10/2023 18:32:28 : Run-AlPipeline start
Determining artifacts to use

Pipeline name                DirectionsNA
Container name               directionsna-bld
Image name                   my
ArtifactUrl                  https://bcartifacts.azureedge.net/sandbox/22.0.54157.55112/us
SasToken                     Not Specified
BcAuthContext                Not Specified
Environment
ReUseContainer               False
KeepContainer                False
Auth                         UserPassword
Credential                   Specified
CompanyName
MemoryLimit                  8G
FailOn                       none
TreatTestFailuresAsWarnings  False
Enable Task Scheduler        False
Assign Premium Plan          False
Install Test Runner          False
Install Test Framework       True
Install Test Libraries       True
Install Perf. Toolkit        False
InstallOnlyReferencedApps    False
generateDependencyArtifact   False
CopySymbolsFromContainer     False
enableCodeCop                True
enableAppSourceCop           True
enableUICop                  True
enablePerTenantExtensionCop  True
doNotPerformUpgrade          False
doNotPublishApps             False
uninstallRemovedApps         False
escapeFromCops               False
doNotBuildTests              False
doNotRunTests                False
doNotRunBcptTests            False
useDefaultAppSourceRuleSet   False
rulesetFile                  C:\Temp\Introduction-to-Test-Automation-in-Business-Central\Supporting-Files\untiltled.ruleset.json
azureDevOps                  False
gitLab                       False
gitHubActions                False
License file                 Not specified
```

References

- https://github.com/microsoft/navcontainerhelper
- https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/developer/devenv-testing-application
- https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/developer/devenv-test-codeunits-and-test-methods
- https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/developer/devenv-creating-handler-methods
- https://learn.microsoft.com/en-us/training/modules/introduction-test-automation/4-run-standard-tests
- https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/developer/properties/devenv-testisolation-property
- https://github.com/dvlprlife/Introduction-to-Test-Automation-in-Business-Central

Thank You!

Please remember to fill out your session survey using the Directions 2023 Mobile App!